SOFTWARE REQUIREMENTS SPECIFICATION

Student Performance Prediction with MLOps

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for the "Student Performance Prediction with MLOps" system. The document describes a machine learning system that predicts academic performance of students and an accompanying MLOps pipeline that automates training, deployment, monitoring, and retraining of that model. It is intended to guide design, implementation, and testing across five delivery phases, and to serve as a shared reference between the developer and any reviewers or collaborators.

1.2 Scope

The system, referred to hereafter as the Student Performance Prediction System (SPPS), will:

- Ingest structured student data (demographic, academic, and behavioral attributes) and produce a prediction of academic performance (pass/fail, grade band, or numeric score).
- Track all experiments, datasets, and model artifacts for full reproducibility.
- Expose the trained model as a versioned REST API for real-time inference.
- Automate build, test, and deployment through a CI/CD pipeline.
- Continuously monitor data drift, prediction drift, and model performance degradation in production.

The following are explicitly out of scope for the version described in this document:

- Building a production-grade student information system (SIS) integration; the project will use static/simulated datasets rather than a live school database.
- Multi-tenant support for multiple institutions.
- Mobile client applications (the system exposes an API and a web dashboard only).

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
MLOps	Machine Learning Operations — practices for deploying and maintaining ML models reliably in production.
SRS	Software Requirements Specification
API	Application Programming Interface
CI/CD	Continuous Integration / Continuous Deployment
DVC	Data Version Control — a tool for versioning datasets and ML pipelines
MLflow	An open-source platform for tracking ML experiments and managing model versions
Drift	A statistically significant change in the distribution of input data or model predictions over time
Concept Drift	A change in the relationship between input features and the target variable
F1-score	Harmonic mean of precision and recall, used to evaluate classification models

Term	Definition
RMSE / MAE	Root Mean Squared Error / Mean Absolute Error — regression evaluation metrics
Model Registry	A centralized store that tracks model versions and their lifecycle stage (staging, production, archived)
Endpoint	A network-addressable URL exposed by the API for a specific function (e.g., /predict)

1.4 References

- UCI Machine Learning Repository – Student Performance Data Set
- Kaggle – Students Performance in Exams Dataset
- Open University Learning Analytics Dataset (OULAD)
- MLflow Documentation — mlflow.org
- DVC Documentation — dvc.org
- Evidently AI Documentation — docs.evidentlyai.com
- FastAPI Documentation — fastapi.tiangolo.com
- IEEE Std 830-1998 — Recommended Practice for Software Requirements Specifications (structural reference)

1.5 Document Overview

Section 2 describes the system at a high level, including its context, users, and constraints. Section 3 presents the system architecture. Section 4 details functional requirements organized by the five project phases. Section 5 covers non-functional requirements. Section 6 specifies external interfaces. Section 7 defines data requirements. Section 8 describes system models and workflows. Section 9 contains appendices, including the tool stack and acceptance criteria.

2. Overall Description

2.1 Product Perspective

SPPS is a self-contained, portfolio-scale system rather than an extension of an existing product. It is composed of five cooperating subsystems, each corresponding to a project phase:

1. Model Development Subsystem — data preparation, feature engineering, training, and experiment tracking.
2. Reproducibility Subsystem — data and pipeline versioning.
3. Serving Subsystem — a containerized REST API exposing predictions.
4. CI/CD Subsystem — automated testing, build, and deployment workflows.
5. Monitoring Subsystem — drift detection, performance tracking, and a dashboard.

These subsystems communicate through shared artifacts (the MLflow Model Registry, the DVC-tracked data store, and structured logs) rather than direct coupling, so each phase can be developed and tested independently before integration.

2.2 Product Functions (Summary)

- Predict a student's academic outcome from input features via a trained ML model.
- Log and compare experiments, and register the best-performing model.

- Version datasets and pipeline stages so any past result can be reproduced exactly.
- Serve predictions over HTTP with input validation and structured error handling.
- Automatically test and deploy new code/model changes on every update.
- Detect data drift and performance decay, and trigger retraining automatically when thresholds are breached.
- Present monitoring results on a dashboard for human review.

2.3 User Classes and Characteristics

User Class	Description
Developer / ML Engineer (primary user)	Builds, trains, deploys, and maintains the system. Needs full visibility into experiments, pipeline runs, and monitoring alerts.
API Consumer	An external application or script that calls the /predict endpoint with student features and receives a prediction. Assumed to be technically competent (sends valid JSON).
Reviewer / Evaluator	Views the monitoring dashboard and documentation to assess system behavior and quality; does not modify the system.

2.4 Operating Environment

- Development OS: Linux or macOS (Windows via WSL2 supported).
- Language/runtime: Python 3.10+.
- Containerization: Docker Engine 24+, docker-compose.
- Orchestration: Prefect (or Apache Airflow) running locally or on a lightweight cloud VM.
- Deployment target: any container-capable host (local machine, a single cloud VM, or a free-tier PaaS such as Render/Railway for demo purposes).
- Version control and CI: GitHub and GitHub Actions.

2.5 Design and Implementation Constraints

- Must use only open-source / free-tier tools so the project can be reproduced without paid licenses.
- The dataset is static/batch in nature; “live” data arrival is simulated by replaying held-out data in batches, not by a real integration with a school system.
- Model serving must respond to a single prediction request in under 1 second on commodity hardware (see Section 5.1).
- All configuration (hyperparameters, thresholds, file paths) must be externalized to YAML config files — no hardcoded values in source code.

2.6 Assumptions and Dependencies

- The chosen dataset is representative enough of real student populations to produce a meaningful demonstration model; it is not assumed to be free of bias, and this limitation will be documented.
- Internet access is available for pulling Docker base images and Python/npm packages during build.
- The system depends on third-party libraries (scikit-learn, MLflow, DVC, FastAPI, Evidently AI) remaining API-compatible with the pinned versions in requirements.txt.

3. System Architecture Overview

The system follows a linear pipeline with feedback: raw data flows through validation and feature engineering into training, producing a versioned model that is deployed behind an API. Production traffic is logged and continuously compared against training data; when drift or performance decay is detected, a retraining job is triggered automatically, closing the loop.

3.1 Architecture Flow

- Data Source → Data Validation → Feature Engineering → Model Training
- Data and Model artifacts are versioned via DVC and MLflow throughout
- CI/CD Pipeline (GitHub Actions) builds, tests, and deploys the API on every change
- Deployed Model (Docker + FastAPI) serves live predictions
- Monitoring Subsystem (Evidently AI + dashboard) watches incoming data and predictions
- Monitoring triggers automated Retraining, which feeds a new candidate model back into the Model Registry

3.2 Design Principles

- Reproducibility first: any historical prediction must be traceable to an exact data version, code commit, and model version.
- Loose coupling: subsystems interact through the Model Registry, versioned datasets, and logs — not direct function calls — so each phase is independently testable.
- Fail safe: a new model is only promoted to production if it outperforms the current production model on a held-out evaluation set; otherwise the system retains the existing model.

4. Phase-wise Functional Requirements

Requirements are grouped by project phase. Each requirement has a unique ID of the form FR-<Phase>-<Number> and a priority: Must (essential to a working demo), Should (important but not blocking), or Could (nice-to-have / stretch goal).

4.1 Phase 1 — Core ML Model Development

Goal: produce a trained, evaluated, and experiment-tracked model that predicts student performance.

ID	Requirement Description	Priority
FR-P1-01	The system shall ingest the chosen dataset (UCI Student Performance, Kaggle Exams, or OULAD) from a local or DVC-tracked source.	Must
FR-P1-02	The system shall perform exploratory data analysis (EDA) and document key findings (distributions, correlations, missing values, class balance).	Must
FR-P1-03	The system shall implement a feature engineering pipeline (encoding categoricals, scaling numerics, handling missing values) as a reusable, callable module — not notebook-only code.	Must
FR-P1-04	The system shall split data into train/validation/test sets using a fixed, documented random seed for reproducibility.	Must
FR-P1-05	The system shall train at least three candidate models (e.g., Logistic Regression, Random Forest, XGBoost/Gradient Boosting).	Must
FR-P1-06	The system shall log every training run's parameters, metrics, and artifacts to MLflow.	Must

ID	Requirement Description	Priority
FR-P1-07	The system shall evaluate models using task-appropriate metrics: Accuracy/Precision/Recall/F1/ROC-AUC for classification, or RMSE/MAE/R^2 for regression.	Must
FR-P1-08	The system shall select the best-performing model against a defined primary metric and register it in the MLflow Model Registry.	Must
FR-P1-09	The system shall expose all training hyperparameters through an external YAML configuration file.	Should
FR-P1-10	The system shall support hyperparameter tuning (e.g., grid search or Optuna) with results tracked in MLflow.	Could

4.2 Phase 2 — Reproducibility & Versioning

Goal: guarantee that any past experiment, dataset state, or model can be exactly reproduced.

ID	Requirement Description	Priority
FR-P2-01	The system shall version raw and processed datasets using DVC, with data files excluded from Git and tracked via .dvc pointer files.	Must
FR-P2-02	The system shall define the end-to-end pipeline (preprocess → train → evaluate) declaratively in a dvc.yaml file with named stages, dependencies, and outputs.	Must
FR-P2-03	The system shall allow `dvc repro` to re-execute only the pipeline stages affected by a change, skipping unchanged stages.	Should
FR-P2-04	The system shall store DVC remote storage configuration (e.g., local remote or cloud bucket) so datasets can be pulled by a fresh clone of the repository.	Must
FR-P2-05	The system shall tag each registered model version in MLflow with the corresponding Git commit hash and DVC data version.	Must
FR-P2-06	The repository shall follow a documented, consistent folder structure (src/, data/, pipelines/, tests/, configs/) as specified in Section 8.3.	Must
FR-P2-07	The system shall include a requirements.txt (or lockfile) pinning exact dependency versions.	Must

4.3 Phase 3 — Model Serving & Deployment

Goal: make the trained model consumable as a live, containerized prediction service.

ID	Requirement Description	Priority
FR-P3-01	The system shall expose a POST /predict endpoint accepting student feature data as JSON and returning a prediction with a confidence/probability score.	Must
FR-P3-02	The system shall validate all incoming request payloads against a defined schema (Pydantic) and return HTTP 422 with a descriptive error for invalid input.	Must

ID	Requirement Description	Priority
FR-P3-03	The system shall expose a GET /health endpoint reporting service status and the currently loaded model version.	Must
FR-P3-04	The system shall load the current “Production” stage model from the MLflow Model Registry at startup.	Must
FR-P3-05	The system shall be packaged as a Docker image with a defined Dockerfile and be runnable via docker-compose alongside its dependencies (e.g., MLflow tracking server).	Must
FR-P3-06	The system shall log every prediction request and response (features, prediction, timestamp, model version) for later monitoring and audit.	Must
FR-P3-07	The system shall support batch prediction via a POST /predict/batch endpoint accepting a list of records.	Should
FR-P3-08	The system shall version its API (e.g., /v1/predict) to allow non-breaking evolution.	Could

4.4 Phase 4 — CI/CD Pipeline

Goal: automatically validate and ship every change with no manual deployment steps.

ID	Requirement Description	Priority
FR-P4-01	The system shall run a GitHub Actions workflow on every push and pull request that installs dependencies and runs the automated test suite.	Must
FR-P4-02	The test suite shall include unit tests for the preprocessing/feature engineering functions.	Must
FR-P4-03	The test suite shall include model tests verifying output shape, output range/type, and a minimum acceptable performance threshold on a fixed test set.	Must
FR-P4-04	The test suite shall include API tests (e.g., via FastAPI's TestClient) covering valid input, invalid input, and the /health endpoint.	Must
FR-P4-05	The workflow shall run a linter/formatter check (e.g., flake8/ruff + black) and fail the build on violations.	Should
FR-P4-06	On a successful build from the main branch, the workflow shall build a Docker image and push it to a container registry (e.g., Docker Hub or GitHub Container Registry).	Must
FR-P4-07	The workflow shall (optionally) deploy the new image to the target host automatically after a successful build.	Should
FR-P4-08	The workflow shall fail the pipeline and prevent deployment if any test stage fails.	Must

4.5 Phase 5 — Monitoring, Drift Detection & Automated Retraining

Goal: detect when the deployed model's assumptions no longer hold, and close the loop by retraining automatically.

ID	Requirement Description	Priority
FR-P5-01	The system shall compute a data drift report (e.g., via Evidently AI) comparing a rolling window of production input data against the training data distribution.	Must
FR-P5-02	The system shall compute prediction drift by comparing the distribution of recent predictions to historical predictions.	Must
FR-P5-03	The system shall track live model performance metrics whenever ground-truth labels become available (e.g., end-of-term actual grades), and compute rolling accuracy/F1/RMSE.	Must
FR-P5-04	The system shall define explicit numeric thresholds (e.g., drift score > 0.3, or accuracy drop > 5 percentage points) that constitute a retraining trigger.	Must
FR-P5-05	The system shall run a scheduled orchestrated job (Prefect/Airflow) that checks drift/performance on a defined interval (e.g., daily).	Must
FR-P5-06	When a retraining trigger fires, the system shall automatically re-run the training pipeline on the latest available data.	Must
FR-P5-07	The system shall evaluate the newly retrained candidate model against the current production model on a common held-out set and shall only promote the candidate if it performs at least as well.	Must
FR-P5-08	The system shall support manual rollback to a previous model version registered in MLflow.	Must
FR-P5-09	The system shall present a dashboard (Streamlit or Grafana) showing current drift score, rolling performance metrics, prediction volume, and retraining history.	Should
FR-P5-10	The system shall send an alert (e.g., log entry, email, or Slack webhook) when a retraining event occurs or when drift exceeds threshold but retraining fails.	Could

5. Non-Functional Requirements

5.1 Performance

- The /predict endpoint shall return a response within 1 second (p95) for a single record on commodity hardware (4 vCPU / 8GB RAM equivalent).
- The training pipeline shall complete end-to-end on the reference dataset in under 15 minutes on the same hardware.

5.2 Scalability

- The serving layer shall be stateless so that multiple API container instances can run behind a load balancer without shared in-memory state.
- The monitoring subsystem shall process at least 10,000 logged predictions per drift-check run without exceeding available memory.

5.3 Reliability & Availability

- The API shall return a graceful HTTP 503 with a clear message if no production model is currently loaded, rather than crashing.
- A failed retraining job shall not affect the currently deployed production model (fail-safe promotion, per FR-P5-07).

5.4 Security

- API input shall be strictly validated to prevent injection or malformed-payload crashes.
- Secrets (registry credentials, cloud tokens) shall be stored in GitHub Actions Secrets / environment variables, never committed to source control.
- Docker images shall be built from minimal, patched base images (e.g., python:3.11-slim) to reduce attack surface.

5.5 Maintainability

- All code shall follow PEP 8 style, enforced by an automated linter in CI (FR-P4-05).
- Configuration values (paths, thresholds, hyperparameters) shall be externalized to YAML files, not hardcoded (Section 2.5).
- Every module shall include docstrings, and the repository shall include a top-level README describing setup and usage.

5.6 Usability

- The monitoring dashboard shall present drift and performance metrics in a way that is interpretable without ML expertise (clear labels, thresholds shown visually).
- API error messages shall be human-readable and indicate exactly which field failed validation.

5.7 Portability

- The entire system (API, tracking server, orchestrator) shall be runnable via a single `docker-compose up` command on any Docker-capable host.

6. External Interface Requirements

6.1 User Interfaces

- A Streamlit (or Grafana) web dashboard displaying drift scores, live performance metrics, prediction volume trends, and retraining event history.
- Auto-generated interactive API documentation via FastAPI's built-in Swagger UI (/docs) and ReDoc (/redoc).

6.2 API Interfaces

Endpoint	Description
POST /predict	Accepts a single student's feature JSON; returns predicted class/score, confidence, and model version used.
POST /predict/batch	Accepts a list of student feature records; returns a list of predictions.
GET /health	Returns service status, loaded model version, and uptime.

Endpoint	Description
GET /metrics	Returns recent prediction volume and basic operational metrics (Prometheus-compatible format optional).

6.3 Software Interfaces

- MLflow Tracking Server — experiment logging and model registry, accessed via its REST API / Python client.
- DVC Remote — local or cloud storage backend for versioned data artifacts.
- Evidently AI — Python library invoked by the monitoring job to compute drift reports.
- GitHub Actions — CI/CD runner, triggered by repository webhooks on push/PR events.
- Container Registry (Docker Hub / GHCR) — stores built Docker images for deployment.

6.4 Communication Interfaces

- All API communication shall use HTTPS in any non-local deployment; HTTP is acceptable for local development only.
- Internal service-to-service communication (API ↔ MLflow) shall occur over the Docker Compose internal network.

7. Data Requirements

7.1 Input Data — Representative Feature Set

Exact fields depend on the dataset selected, but the system shall support at minimum the following categories of input features:

Category	Example Fields
Demographic	age, gender, family size, parental education/occupation
Academic history	previous grades, number of past failures, study time per week
Behavioral / engagement	attendance rate, absences, participation in extracurriculars, VLE clicks (if using OULAD)
Socioeconomic	internet access at home, family educational support, travel time to school
Target variable	final grade (numeric), or pass/fail label, or grade band (A–F)

7.2 Data Quality Requirements

- Missing values shall be handled explicitly (imputation or documented row exclusion) — never silently dropped without logging.
- Categorical values shall be validated against an allowed value set at the API boundary; unrecognized categories shall be rejected with a clear error.
- The training/validation/test split shall preserve target class balance (stratified split) for classification tasks.

7.3 Data Retention

- Logged prediction requests/responses shall be retained for a minimum of 90 days to support drift analysis and audit.
- Raw and processed training datasets shall be retained indefinitely under DVC version control.

8. System Models

8.1 Primary Use Cases

ID	Use Case
UC-1	Train and register a new model — Actor: ML Engineer. Engineer runs the training pipeline; system logs the run to MLflow and registers the model if it meets the acceptance threshold.
UC-2	Request a prediction — Actor: API Consumer. Consumer sends student features to /predict; system validates input, runs inference, logs the request, and returns a result.
UC-3	Review monitoring dashboard — Actor: Reviewer. Reviewer opens the dashboard to inspect drift scores and recent performance.
UC-4	Automated retraining — Actor: Orchestrator (system-initiated). Scheduled job detects drift/performance breach, triggers retraining, evaluates the candidate, and promotes or discards it.
UC-5	Roll back a model — Actor: ML Engineer. Engineer manually demotes the current production model and promotes a previous registered version.

8.2 Core Workflow — Automated Retraining Loop

6. Scheduled monitoring job pulls a recent window of logged production predictions.
7. Evidently AI computes a data-drift report against the reference (training) dataset.
8. If drift score or performance metric breaches the configured threshold (FR-P5-04), a retraining trigger is emitted.
9. The orchestrator invokes the training pipeline (Phase 1 logic) on the latest available labeled data.
10. The new candidate model is evaluated on a fixed held-out benchmark set.
11. If the candidate model's primary metric is greater than or equal to the current production model's metric, it is promoted to “Production” in the MLflow Model Registry; otherwise it is archived as “Staging” and the existing model remains live.
12. The deployed API reloads the production model (or continues serving the current one if no promotion occurred).
13. The event (triggered / promoted / rejected) is recorded and surfaced on the dashboard.

8.3 Repository Structure

The project repository shall follow this structure so that each phase's artifacts are clearly located:

```
student-performance-mlops/
├── data/                   # raw + processed data (DVC tracked)
├── src/
│   ├── data/               # ingestion, validation
│   ├── features/           # feature engineering
│   ├── models/             # train.py, evaluate.py
│   └── api/                # FastAPI app
├── pipelines/              # Prefect/Airflow DAGs
├── monitoring/             # Evidently configs, dashboard
├── tests/                  # unit, model, and API tests
├── configs/                # YAML configuration files
├── dvc.yaml                # pipeline stage definitions
├── Dockerfile
├── docker-compose.yml
├── .github/workflows/      # CI/CD pipeline definitions
└── requirements.txt
```

9. Appendices

9.1 Recommended Tool Stack

Component	Tool
Experiment Tracking	MLflow
Data / Pipeline Versioning	DVC
Model Serving	FastAPI
Containerization	Docker / docker-compose
Orchestration	Prefect (or Apache Airflow)
CI/CD	GitHub Actions
Monitoring / Drift Detection	Evidently AI
Dashboard	Streamlit (or Grafana)
Model Registry	MLflow Model Registry
Testing	pytest

9.2 Acceptance Criteria Summary (per phase)

Phase	Acceptance Criteria
Phase 1	At least 3 models trained and logged to MLflow; best model registered; evaluation report produced.
Phase 2	`dvc repro` reproduces the full pipeline from a clean clone; dvc.yaml and .dvc files committed; model versions traceable to Git + DVC commit.
Phase 3	`docker-compose up` serves a working /predict endpoint returning correct predictions with <1s latency; invalid input returns HTTP 422.

Phase	Acceptance Criteria
Phase 4	A pull request with a failing test blocks merge/deploy; a passing push to main builds and publishes a Docker image automatically.
Phase 5	Simulated drifted data batch triggers an automatic retraining run, and the dashboard reflects the before/after metrics and promotion decision.

9.3 Risks and Mitigations

Risk	Mitigation
Small dataset limits realistic drift simulation	Split data by time/demographic slice to artificially create distribution shift for demo purposes; document this as simulated, not organic, drift.
Overfitting to a small benchmark	Use cross-validation during Phase 1 and a held-out test set that is never used for hyperparameter tuning.
CI/CD pipeline flakiness (external service timeouts)	Pin dependency versions; add retries/timeouts in workflow steps; keep the tracking server as a lightweight local service where possible.
Model bias from demographic/socioeconomic features	Document known dataset limitations; consider a fairness check (e.g., performance parity across gender or family background) as a stretch goal.

9.4 Out-of-Scope / Future Work

- Real-time streaming ingestion from a live student information system.
- Multi-institution / multi-tenant deployment with per-tenant models.
- A/B testing framework for comparing production model variants on live traffic.
- Formal fairness/bias auditing pipeline beyond basic parity checks.